

ANNA UNIVERSITY, CHENNAI

Department of Computer Science and Engineering

ASSIGNMENT

Data Types, Vectors, Data Structures and Packages in R

Name	Santhosh K
Register Number	2023103552
Batch	CSE 'N' Batch
Semester	VII Semester
Subject Code	CS23305
Subject Title	Business Analytics

1. Data Types in R

Every object that R stores has an underlying data type, sometimes called its "mode". Because R is dynamically typed, a variable does not need its type declared in advance - the type is inferred automatically from whatever value is assigned to it. The functions `class()` and `typeof()` let a programmer inspect this type at any point in a script.

Data Type	Meaning	Sample Assignment	class() Result
Double / Numeric	Default type for decimals	<code>score <- 91.5</code>	"numeric"
Integer	Whole numbers, needs L suffix	<code>count <- 40L</code>	"integer"
Character	Text / string data	<code>city <- "Chennai"</code>	"character"
Logical	Boolean TRUE / FALSE	<code>passed <- TRUE</code>	"logical"
Complex	Real + imaginary part	<code>z <- 4 + 3i</code>	"complex"
Raw	Raw byte storage	<code>b <- as.raw(15)</code>	"raw"

1.1 typeof() versus class()

`typeof()` reports the low-level storage mode that R keeps internally, whereas `class()` reports the higher-level object class that R's method-dispatch system uses. For plain atomic values the two usually agree, but they diverge for composite objects such as matrices.

```
code <- 40L
typeof(code)      # "integer" -> internal storage mode
class(code)       # "integer" -> object class
is.numeric(code)  # TRUE

mtx <- matrix(1:6, nrow = 2)
typeof(mtx)       # "integer"
class(mtx)        # "matrix" "array" -> class differs from typeof
```

1.2 Converting Between Types (Coercion)

R can convert a value from one type to another either explicitly, through a family of `as.*()` functions, or implicitly, whenever values of different types are forced to live inside the same atomic vector.

```
# explicit conversion
txt <- as.character(48)    # "48"
num <- as.numeric("6.75") # 6.75
flag <- as.logical("FALSE") # FALSE
whole <- as.integer(9.8)   # 9 (fraction is dropped, not rounded)
```

When several types are mixed inside one call to `c()`, R silently promotes every element to the single most flexible type present, following the order logical → integer → numeric → character.

```
c(TRUE, 5L)      # 5 1 -> promoted to integer
c(5L, 2.5)       # 5.0 2.5 -> promoted to numeric
c(2.5, "x")      # "2.5" "x" -> promoted to character (highest in the chain)
```

1.3 Special Constants

R defines four special constants for representing missing, undefined or infinite results, and each one behaves a little differently in expressions.

- **NA** - marks a single missing value; it can stand in for any type inside a vector.
- **NULL** - represents the total absence of a value or an empty object.
- **NaN** - "Not a Number", produced by an undefined operation such as $0/0$.
- **Inf / -Inf** - positive or negative infinity, e.g. the result of $5/0$.

```
is.na(NA)      # TRUE
is.null(NULL)  # TRUE
0 / 0          # NaN
7 / 0          # Inf
length(NULL)   # 0
```

2. Vectors in R

The vector is the simplest and most heavily used data structure in R: an ordered, one-dimensional collection whose elements must all share a single data type. Nearly every other R structure - matrices, arrays, and data frames - is really a vector with extra structure layered on top of it. Even a lone value like `n <- 7` is, internally, a vector of length one.

2.1 Building Vectors

```
# c() joins individual values into one vector
ages    <- c(21, 24, 19, 30)      # numeric vector
cities  <- c("Chennai", "Pune", "Kochi") # character vector
flags   <- c(TRUE, FALSE, FALSE)  # logical vector

# generating a run of numbers
run1 <- 1:8                      # 1 2 3 4 5 6 7 8
run2 <- seq(5, 50, by = 5)       # 5 10 15 ... 50

# repeating a pattern
pat <- rep(c(0, 1), times = 4)   # 0 1 0 1 0 1 0 1

# a placeholder vector of zeros
blank <- vector("numeric", length = 6) # 0 0 0 0 0 0
```

2.2 Naming Elements

Giving names to the elements of a vector makes code more self-explanatory and allows values to be pulled out by label instead of by numeric position.

```
scores <- c(DBMS = 78, OS = 84, CN = 91)
names(scores) # "DBMS" "OS" "CN"
scores["OS"]  # 84
scores[[2]]   # 84 (double brackets strip away the name, giving the raw value)
```

2.3 Indexing and Slicing

Unlike many languages, R numbers vector positions starting at 1, not 0. Square brackets support several ways of pulling out a subset: by position, by a vector of positions, by excluding positions, or by a logical condition.

```
v <- c(4, 9, 15, 22, 30)

v[1]          # 4          -> first element
v[c(2, 4)]    # 9 22      -> elements 2 and 4
v[-1]         # 9 15 22 30 -> everything except the first element
v[v > 15]     # 22 30     -> logical / conditional filter
```

2.4 Vectorised Arithmetic and Recycling

Arithmetic operators in R act element-by-element rather than on the vector as a whole. If two vectors involved in an operation are of unequal length, R silently repeats ("recycles") the shorter one until the lengths line up - a behaviour worth watching for, since it can hide bugs when lengths are not multiples of each other.

```
p <- c(2, 4, 6, 8)
q <- c(10, 10, 10, 10)
p + q                # 12 14 16 18

c(1, 2, 3, 4) * c(1, 2) # 1 4 3 8    (c(1,2) recycled twice)

length(p)           # 4
sum(p)              # 20
mean(p)             # 5
sort(q, decreasing = FALSE) # 10 10 10 10
```

2.5 Handy Built-in Vector Functions

```
x <- c(7, 3, NA, 12, 3, 1)

which(x > 5)          # 1 4          -> positions satisfying the condition
which.max(x)          # 4            -> position of the largest value
unique(x)             # 7 3 NA 12 1  -> drops repeated values
rev(x)               # reverses the element order
is.na(x)             # FALSE FALSE TRUE FALSE FALSE FALSE
sum(x, na.rm = TRUE) # 26            -> NA skipped while totalling
seq_along(x)         # 1 2 3 4 5 6  -> index positions of x
```

3. Data Structures in R

R offers a compact family of core structures. They differ along two axes: how many dimensions they organise data into (one, two, or many), and whether every element must share one type (homogeneous) or may mix types (heterogeneous).

Dimension	Homogeneous (one type)	Heterogeneous (mixed types)
1-D	Vector	List
2-D	Matrix	Data Frame
n-D	Array	-

A related structure, the **factor**, is a special vector built to hold categorical data drawn from a fixed set of allowed values (its "levels").

3.1 Vector (recap)

As introduced in Section 2, a vector is the one-dimensional, single-type building block that every other structure below is ultimately assembled from.

```
v <- c(3, 6, 9, 12)
class(v)      # "numeric"
str(v)        # num [1:4] 3 6 9 12
```

3.2 Matrix

A matrix arranges a single-type collection of values into rows and columns - effectively a vector with two dimensions attached. Every column must contain the same number of elements.

```
mx <- matrix(1:6, nrow = 3, ncol = 2)
#      [,1] [,2]
# [1,]  1   4
# [2,]  2   5
# [3,]  3   6

dim(mx)      # 3 2
mx[2, 1]     # 2 -> row 2, column 1
mx[, 2]      # 4 5 6 -> the whole second column
mx[3, ]      # 3 6 -> the whole third row
t(mx)        # transpose
mx %*% t(mx) # matrix multiplication
```

3.3 Array

An array extends the matrix idea to three or more dimensions while still requiring a single data type throughout. It can be pictured as a stack of same-sized matrices layered one behind another.

```
cube <- array(1:30, dim = c(3, 2, 5)) # 3 rows x 2 cols x 5 layers
dim(cube)      # 3 2 5
cube[2, 1, 4]  # element at row 2, column 1, layer 4
cube[, , 1]    # the entire first 3x2 layer
```

3.4 List

A list is an ordered container that, unlike a vector, may hold elements of different types and even different structures - a list can hold vectors, data frames, or further lists nested inside it.

```
emp <- list(name = "Santhosh", age = 22, ratings = c(4.5, 4.8, 4.2))

emp$name          # "Santhosh"
emp[[3]]          # 4.5 4.8 4.2 -> [[ ]] pulls out the element itself
emp[3]            # a one-item list wrapping that vector, not the vector itself
emp$ratings[2]    # 4.8
length(emp)       # 3 -> number of top-level elements
```

3.5 Data Frame

The data frame is the everyday structure for spreadsheet-like, tabular data. Internally it is a list of equal-length vectors that R displays as a table, and - unlike a matrix - each column is free to hold its own data type.

```
emp_df <- data.frame(
  Name   = c("Divya", "Karthik", "Meena"),
  Salary = c(52000, 61000, 48000),
  Active = c(TRUE, TRUE, FALSE)
)

str(emp_df)          # structure/summary of the frame
nrow(emp_df); ncol(emp_df) # 3 rows, 3 columns
emp_df$Salary        # 52000 61000 48000
emp_df[emp_df$Salary > 50000, ] # rows earning above 50000
emp_df[order(-emp_df$Salary), ] # sorted by Salary, highest first
```

3.6 Factor

A factor stores categorical variables - such as department, grade, or region - as a fixed set of labelled levels rather than free-form text, which makes grouping and summarising such data far more efficient.

```
dept <- factor(c("CSE", "ECE", "CSE", "IT"), levels = c("CSE", "ECE", "IT"))
levels(dept)      # "CSE" "ECE" "IT"
nlevels(dept)     # 3
table(dept)       # frequency count per level
as.integer(dept)  # 1 2 1 3 -> underlying integer codes
```

3.7 Structures Compared

Structure	Dimensions	Data Types	Typical Use
Vector	1-D	Single type	Simple ordered list of values
Matrix	2-D	Single type	Numeric / linear-algebra work
Array	n-D	Single type	Multi-way numeric data
List	1-D (tree)	Mixed types	Bundling unrelated results
Data Frame	2-D	Mixed per column	Real-world tabular datasets
Factor	1-D	Categorical labels	Grouping / classification

4. R Packages

An R package bundles together functions, compiled routines, sample datasets and documentation into a single shareable unit. Packages are what let R grow beyond its base functionality: rather than coding every statistical or plotting routine from first principles, a user installs a package someone else has already written and immediately gains access to its functions. CRAN alone now hosts well over 20,000 community-contributed packages spanning almost every corner of statistics, data science and visualisation.

4.1 Categories of Packages

Depending on how a package reaches a user's computer, R packages fall into three broad groups:

- **Base packages** - ship with every copy of R and load automatically at startup (e.g. base, stats, utils, graphics).
- **Recommended packages** - bundled together with the R installer, but versioned and updated on their own separate schedule (e.g. MASS, Matrix, survival).
- **Contributed packages** - authored by the wider R community and must be installed manually before use (e.g. dplyr, ggplot2, caret).

4.2 Where Packages Come From

- **CRAN** (Comprehensive R Archive Network) - the official and most widely used package repository; every submission passes an automated check before it is published.
- **Bioconductor** - a repository dedicated to bioinformatics and genomic-data-analysis packages.
- **GitHub** - a common home for in-development or experimental packages, typically installed with `devtools::install_github()` or `remotes::install_github()`.

4.3 Installing, Loading, Updating and Removing

A package is downloaded and installed onto disk only once, but it must be loaded with `library()` into every fresh R session where its functions are needed.

```
# install once from CRAN
install.packages("ggplot2")

# load into the current session (repeat every session)
library(ggplot2)    # throws an error if the package isn't installed
require(ggplot2)    # returns FALSE instead of erroring out

# see everything currently loaded
search()

# bring every installed package up to its latest version
update.packages()

# remove a package that is no longer needed
remove.packages("ggplot2")

# read the documentation for a package
help(package = "ggplot2")
vignette("ggplot2")  # long-form usage guide, where one is provided
```

4.4 A Few Widely Used Packages

- **dplyr** - fast, consistent verbs for data manipulation (filter, select, mutate, summarise).
- **ggplot2** - a grammar-of-graphics engine for building layered, publication-quality charts.
- **tidyr** - reshapes messy data into a clean, consistent row/column layout.
- **readr / readxl** - quick, friendly functions for reading CSV and Excel files into R.
- **caret** - a single, unified interface for training and evaluating ML models.
- **shiny** - builds interactive web applications directly from R code.
- **forecast** - tools for time-series analysis and business forecasting.
- **lubridate** - makes working with dates and times far less painful.

4.5 Anatomy of a Package

Every package follows a standard folder layout so that R's build tools can recognise and compile it correctly:

```
mypkg/  
  DESCRIPTION -> metadata: package name, version, author, dependencies  
  NAMESPACE  -> declares which functions are exported / imported  
  R/          -> folder holding the actual .R source files  
  man/        -> help-page documentation for each exported function  
  data/       -> optional sample datasets shipped with the package  
  vignettes/  -> optional long-form tutorials / usage guides  
  tests/      -> automated tests that check the package behaves correctly
```

4.6 Inspecting Session and Package Details

```
sessionInfo()      # R version, OS, and every loaded package + its version  
packageVersion("dplyr") # version number of a single named package  
installed.packages() # a table listing every package installed locally
```

-- End of Assignment --